JavaScript is disabled on your browser.

compact1, compact2, compact3

java.io

Class FilterInputStream

java.lang.Object
    java.io.InputStream
        java.io.FilterInputStream

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Direct Known Subclasses:

`BufferedInputStream`, `CheckedInputStream`, `CipherInputStream`, `DataInputStream`, `DeflaterInputStream`, `DigestInputStream`, `InflaterInputStream`, `LineNumberInputStream`, `ProgressMonitorInputStream`, `PushbackInputStream`

```
public class FilterInputStream
extends InputStream
```

A `FilterInputStream` contains some other input stream, which it uses as its basic source of data, possibly transforming the data along the way or providing additional functionality. The class `FilterInputStream` itself simply overrides all methods of `InputStream` with versions that pass all requests to the contained input stream. Subclasses of `FilterInputStream` may further override some of these methods and may also provide additional methods and fields.

Since:

`JDK1.0`

Field Summary

Fields

Modifier and Type	Field	Description

protected **InputStream** **in**		The input stream to be filtered.

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	**FilterInputStream**(**InputStream** in)	Creates a `FilterInputStream` by assigning the argument `in` to the field `this.in` so as to remember it for later use.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
int	**available**()	Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next caller of a method for this input stream.
void	**close**()	Closes this input stream and releases any system resources associated with the stream.
void	**mark**(int readlimit)	Marks the current position in this input stream.
boolean	**markSupported**()	Tests if this input stream supports the `mark` and `reset` methods.
int	**read**()	Reads the next byte of data from this input stream.
int	**read**(byte[] b)	

		Reads up to `byte.length` bytes of data from this input stream into an array of bytes.
`int`	**`read`**`(byte[] b, int off, int len)`	Reads up to `len` bytes of data from this input stream into an array of bytes.
`void`	**`reset`**`()`	Repositions this stream to the position at the time the `mark` method was last called on this input stream.
`long`	**`skip`**`(long n)`	Skips over and discards `n` bytes of data from the input stream.

Methods inherited from class java.lang.Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Detail

in

```
protected volatile InputStream in
```

The input stream to be filtered.

Constructor Detail

FilterInputStream

```
protected FilterInputStream(InputStream in)
```

Creates a `FilterInputStream` by assigning the argument `in` to the field `this.in` so as to remember it for later use.

Parameters:

`in - the underlying input stream, or null if this instance is to be created without an underlying stream.`

Method Detail

read

```
public int read()
         throws IOException
```

Reads the next byte of data from this input stream. The value byte is returned as an `int` in the range `0` to `255`. If no byte is available because the end of the stream has been reached, the value `-1` is returned. This method blocks until input data is available, the end of the stream is detected, or an exception is thrown.

This method simply performs `in.read()` and returns the result.

Specified by:

`read` in class `InputStream`

Returns:

the next byte of data, or -1 if the end of the stream is reached.

Throws:

`IOException` - if an I/O error occurs.

See Also:

`in`

read

```
public int read(byte[] b)
         throws IOException
```

Reads up to `byte.length` bytes of data from this input stream into an array of bytes. This method blocks until some input is available.

This method simply performs the call `read(b, 0, b.length)` and returns the result. It is important that it does *not* do `in.read(b)` instead; certain subclasses of `FilterInputStream` depend on the implementation strategy actually used.

Overrides:

`read` in class `InputStream`

Parameters:

b - the buffer into which the data is read.

Returns:

the total number of bytes read into the buffer, or -1 if there is no more data because the end of the stream has been reached.

Throws:

IOException - if an I/O error occurs.

See Also:

read(byte[], int, int)

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads up to len bytes of data from this input stream into an array of bytes. If len is not zero, the method blocks until some input is available; otherwise, no bytes are read and 0 is returned.

This method simply performs in.read(b, off, len) and returns the result.

Overrides:

read in class InputStream

Parameters:

b - the buffer into which the data is read.

off - the start offset in the destination array b

len - the maximum number of bytes read.

Returns:

the total number of bytes read into the buffer, or -1 if there is no more data because the end of the stream has been reached.

Throws:

NullPointerException - If b is null.

IndexOutOfBoundsException - If off is negative, len is negative, or len is greater than b.length - off

IOException - if an I/O error occurs.

See Also:

in

skip

```
public long skip(long n)
          throws IOException
```

Skips over and discards n bytes of data from the input stream. The skip method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly 0. The actual number of bytes skipped is returned.

This method simply performs `in.skip(n)`.

Overrides:

`skip` in class `InputStream`

Parameters:

`n - the number of bytes to be skipped.`

Returns:

`the actual number of bytes skipped.`

Throws:

`IOException - if the stream does not support seek, or if some other I/O error occurs.`

available

```
public int available()
              throws IOException
```

Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next caller of a method for this input stream. The next caller might be the same thread or another thread. A single read or skip of this many bytes will not block, but may read or skip fewer bytes.

This method returns the result of `in`.available().

Overrides:

`available` in class `InputStream`

Returns:

`an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking.`

Throws:

`IOException - if an I/O error occurs.`

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream. This method simply performs `in.close()`.

Specified by:

`close` in interface `Closeable`

Specified by:

close in interface AutoCloseable

Overrides:

close in class InputStream

Throws:

IOException - if an I/O error occurs.

See Also:

in

mark

```
public void mark(int readlimit)
```

Marks the current position in this input stream. A subsequent call to the reset method repositions this stream at the last marked position so that subsequent reads re-read the same bytes.

The readlimit argument tells this input stream to allow that many bytes to be read before the mark position gets invalidated.

This method simply performs in.mark(readlimit).

Overrides:

mark in class InputStream

Parameters:

readlimit - the maximum limit of bytes that can be read before the mark position becomes invalid.

See Also:

in, reset()

reset

```
public void reset()
           throws IOException
```

Repositions this stream to the position at the time the mark method was last called on this input stream.

This method simply performs in.reset().

Stream marks are intended to be used in situations where you need to read ahead a little to see what's in the stream. Often this is most easily done by invoking some general parser. If the stream is of the type handled by the parse, it just chugs along happily. If the stream is not of that type, the parser should toss an exception when it fails. If this happens within readlimit bytes, it allows the outer code to reset the stream and try another parser.

Overrides:

`reset` in class `InputStream`

Throws:

`IOException` - if the stream has not been marked or if the mark has been invalidated.

See Also:

`in`, `mark(int)`

markSupported

```
public boolean markSupported()
```

Tests if this input stream supports the `mark` and `reset` methods. This method simply performs `in.markSupported()`.

Overrides:

`markSupported` in class `InputStream`

Returns:

`true` if this stream type supports the mark and reset method; `false` otherwise.

See Also:

`in`, `InputStream.mark(int)`, `InputStream.reset()`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.